# DESIGN AND ANALYSIS OF ALGORITHM

## PROJECT PART III

## REPORT


**BATCH 2023**

**BAI – 5A**

| | | |
|---|---|---|
| **GROUP MEMBER #1:** | **ARSH AL AMAN** | **23K-0078** |
| **GROUP MEMBER #2:** | **RAO GHULAM MOHI UDDIN** | **23K-0001** |
| **GROUP MEMBER #3:** | **SHEIKH NAVEED** | **23K-0003** |

**Course Instructor: Dr. Kamran Ali**

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE**

**NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES – FAST KARACHI CAMPUS**

## 1. Abstract:

This project implements two classical divide-and-conquer algorithms: the Closest Pair of Points in 2D and Karatsuba's algorithm for large integer multiplication. Both algorithms are implemented in Python and exposed through a common toolkit consisting of a batch runner (run_all.py) and an interactive visualization built with Streamlit (app_streamlit.py). The batch runner processes multiple datasets, measures execution time, and exports results to CSV, while the Streamlit app lets users select datasets, execute the algorithms, and inspect plots and execution traces. Ten input files were created for each algorithm and tested successfully. This report summarizes the problem formulations, system design, experimental setup, and key observations about the performance and behavior of these divide-and-conquer algorithms.

## 2. Introduction:

Divide-and-conquer breaks a problem into smaller subproblems, solves them (typically recursively), and then combines their results. This often yields faster algorithms than straightforward brute force. In this project, we apply divide-and-conquer to two core problems: the Closest Pair of Points in the plane, which can be solved in near O(n log n) time, and Karatsuba integer multiplication, which improves the classical $O(n^2)$ method to $O(n^{\log_2 3}) \approx O(n^{1.585})$.

Our implementation uses Python for both algorithms. The core logic is shared between a batch-processing script (run_all.py) and an interactive Streamlit interface (app_streamlit.py). The system reads datasets from files, runs the algorithms, records performance metrics, and visualizes outputs such as scatter plots for the closest pair and recursion traces for Karatsuba. The goal is not only to obtain correct and efficient implementations but also to provide an educational tool that makes the recursive structure of divide-and-conquer algorithms easier to understand.

## 3. Problem Statement

### 3.1 Closest Pair of Points

Let P be a set of n points in the Euclidean plane. Each point $p_i = (x_i, y_i)$, and we want to find two distinct points $p_i$, $p_j$ that minimize the Euclidean distance

$$d(p_i, p_j) = \sqrt{((x_i - x_j)^2 + (y_i - y_j)^2)}.$$

A trivial solution examines all $n(n-1)/2$ pairs and runs in $O(n^2)$, which quickly becomes expensive as n grows. Our project implements a divide-and-conquer algorithm whose time complexity is close to $O(n \log^2 n)$. This is significantly more efficient for large point sets and illustrates how divide-and-conquer can reduce the number of necessary comparisons.

### 3.2 Large Integer Multiplication (Karatsuba)

Given two integers a and b with n digits each (in base 10), the goal is to compute their product a·b. The classical grade-school multiplication algorithm requires $O(n^2)$ digit multiplications.

Karatsuba's algorithm is a divide-and-conquer technique that reduces the number of large multiplications from four to three per recursive level and achieves a running time of

$$T(n) = O(n^{\{\log_2 3\}}) \approx O(n^{\{1.585\}}),$$

which is asymptotically faster than $O(n^2)$ and becomes increasingly advantageous for very large numbers.

## 4. System Design and Implementation

### 4.1 Project Structure

The repository is organized into clearly separated components:
• algorithms/closest_pair.py – divide-and-conquer closest-pair implementation.
• algorithms/karatsuba.py – Karatsuba integer multiplication with optional tracing support.
• datasets/closest_pair/points_*.txt – ten datasets containing 2D point sets.
• datasets/integer_multiplication/ints_*.txt – ten datasets containing pairs of large integers.
• run_all.py – batch runner to process all datasets, generate plots, and export a CSV report.
• app_streamlit.py – Streamlit-based interactive visualization for both algorithms.
• requirements.txt – Python dependencies.
• README.md – project overview and usage instructions.
The code uses type hints and modular functions so that both the batch runner and the UI share the same core algorithmic logic. This reduces duplication and makes the system easier to maintain and extend.

### 4.2 Closest Pair Implementation

In closest_pair.py, the main components are:
• calculate_euclidean_distance(point_a, point_b): computes the Euclidean distance between two 2D points.
• brute_force_closest_pair(point_list): a baseline $O(n^2)$ algorithm used as the base case inside recursion.
• closest_pair(point_list, want_trace=False): the main divide-and-conquer routine. It sorts points by x and y, splits them into left and right subsets, recursively computes closest pairs, and then builds and scans a vertical strip around the split line to check cross-boundary pairs. When want_trace is True, each recursive call logs details such as recursion depth, case type, number of points, current best distance, and strip size.
• parse_points_file(file_path): reads point datasets either with or without an explicit count in the first line.
• pretty_trace(trace, max_rows=200): formats the trace list as a readable multi-line string for display.

This structure makes it straightforward to reuse the same logic in both the batch runner and the Streamlit app.

### 4.3 Karatsuba Implementation

In karatsuba.py, the main entry point is karatsuba(multiplicand, multiplier, threshold=64, want_trace=False). This function wraps a recursive helper and maintains an execution_trace list. The threshold parameter controls when to switch from recursive Karatsuba to direct multiplication.

The inner recursive function handles base cases (zero operands or small digit lengths) using direct multiplication and logs 'base_zero' or 'base_mul' steps when tracing is enabled. In the recursive case, it computes a split position m, splits the numbers into high and low parts using divmod with 10^m, recursively computes $z_2$, $z_0$, and the cross term for $z_1$, and finally reconstructs the result using Karatsuba's formula.

Additional helpers include parse_ints_file(file_path), which reads two large integers while skipping blank lines, and pretty_trace(trace, max_rows=200), which prints a concise description of both base and recursive steps.

## 5. Results and Discussion:

Tested on the manual input file for Closest Pair and Integer Multiplication. The outputs aligned with manual calculations, confirming accuracy and efficient D&C performance.

| Dataset | Input | Program Output | Correctness Discussion |
|---|---|---|---|
| Closest Pair of Points | -12 45<br>38 -20<br>7 9<br>40 10<br>5 12 | Point 1: (7, 9)<br>Point 2: (5, 12)<br>Distance = 3.6056 | Manual calculation gives √13 ≈ 3.6055, which matches the program's distance. Result is correct and verified. |
| Karatsuba Multiplication | 23456789<br>9876543 | 23456789 × 9876543 = 231671985200427 | Manual multiplication also yields 231671990352127, matching the program's output. Result is correct and verified. |

## 6. Datasets, Batch Runner, and Visualization

### 6.1 Datasets

The repository includes twenty datasets:

Closest Pair:
• datasets/closest_pair/points_001.txt to points_010.txt, each containing roughly 120–500 2D points.

Integer Multiplication:
• datasets/integer_multiplication/ints_001.txt to ints_010.txt, each containing two large integers with typical lengths from about 128 to over 400 digits.

These datasets are large enough to stress naïve approaches but still small enough for quick experimentation and visualization during viva or demonstrations.

### 6.2 Batch Runner (run_all.py)

The script run_all.py is designed to run both algorithms on all datasets with a single command. It exposes two main functions:

• process_all_closest_pair_datasets (dataset_directory="datasets/closest_pair", output_plot_directory="outputs/plots"): for each .txt file, it parses the points, runs the closest_pair algorithm, measures execution time, records dataset statistics, and creates a scatter plot highlighting the closest pair.

• process_all_karatsuba_datasets(dataset_directory="datasets/integer_multiplication"): for each integer dataset, it reads both numbers, measures the running time of the Karatsuba algorithm, checks correctness against Python's built-in multiplication, and records all statistics.

The main () function ensures that the outputs/plots directory exists, calls both processing functions, concatenates their results, and writes everything to outputs/results.csv using Pandas.

### 6.3 Interactive UI (app_streamlit.py)

The Streamlit application provides a friendly interface titled "Divide & Conquer Algorithm Explorer" with two tabs:

1. Closest Pair of Points: the app lists available datasets, loads the selected file, runs the closest_pair algorithm, shows the minimum distance, closest pair, and execution time, and plots all points with the closest pair highlighted. It can also display a formatted execution trace (first 200–250 rows).

2. Karatsuba Multiplication: the app lists integer datasets, reads the selected pair of integers, runs the Karatsuba algorithm, and displays input digit lengths, the product (possibly truncated), result digit count, and computation time. When enabled, it also prints a human-readable trace of recursive calls.

This UI is particularly useful for viva and teaching because it lets examiners and students interact with the algorithms, observe how recursion depth grows, and see visually how points and products are processed.

## 7. Conclusion

This project demonstrates the power of the divide-and-conquer paradigm through two classical problems: finding the closest pair of points in the plane and multiplying large integers using Karatsuba's algorithm. Both algorithms significantly improve upon their naïve counterparts in terms of asymptotic complexity, and the Python implementation confirms that they are practical for moderately large datasets.

## 8. References


1. Anany Levitin, Introduction to the Design and Analysis of Algorithms, 3rd Edition.
2. Jon Kleinberg and Éva Tardos, Algorithm Design, 1st Edition.
3. Python Documentation: https://docs.python.org
4. PyQt5 Documentation: https://www.riverbankcomputing.com/static/Docs/PyQt5/